Network Traffic Profiler Dashboard

UNIT TEST DOCUMENTATION

1. Overview / Purpose (Amelia)

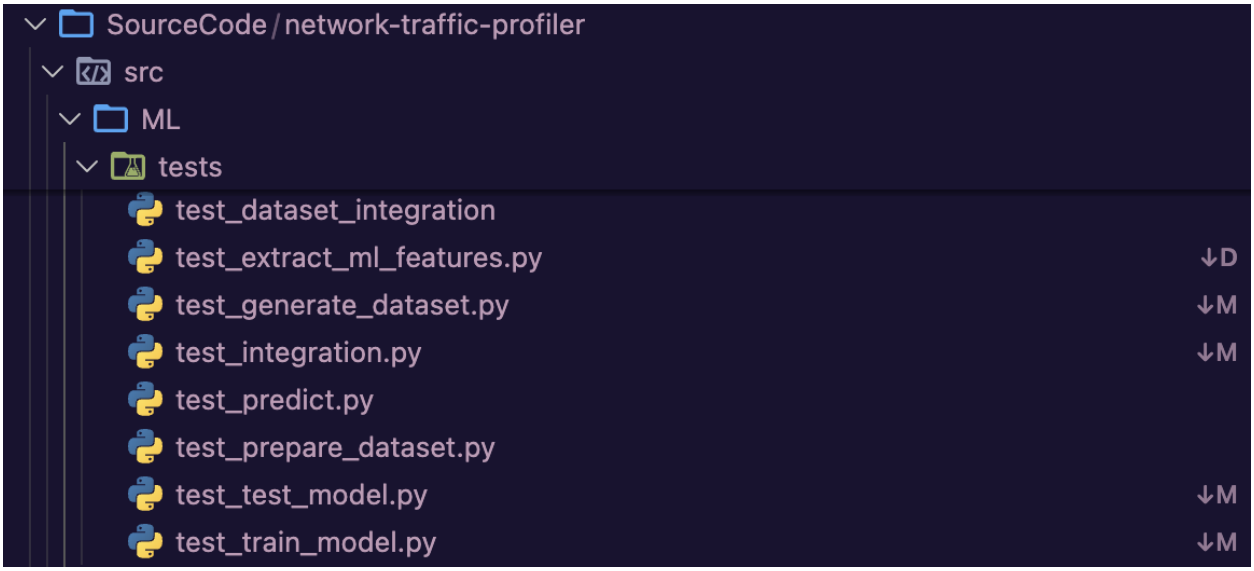
- Briefly explain what the unit tests cover.
- Describe the overall goals & objectives

---

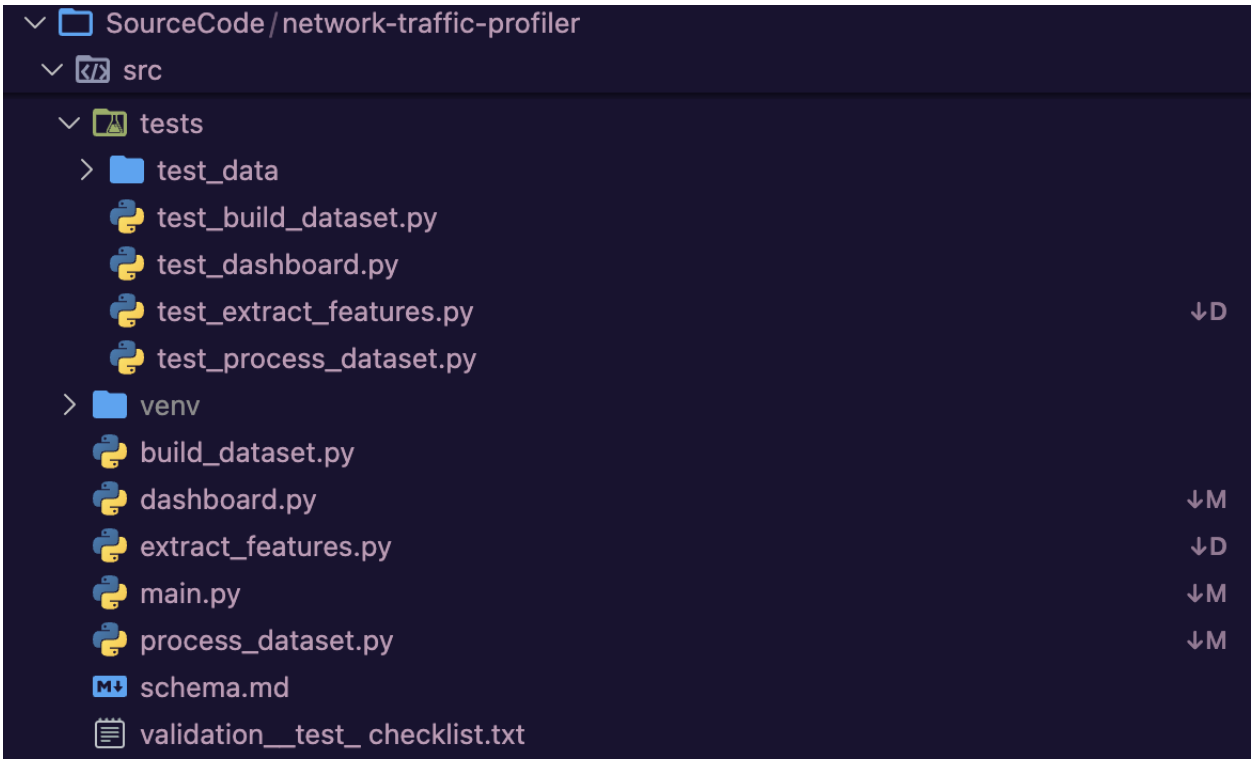
2. Test Files/ Folder Structure (Sophia)

Tests are contained in their own /tests folders and are split up between the ML pipeline and the main (dashboard) pipeline. Each test begins with the keyword test_, the common naming convention for tests written with the pytest library.

src//ML/tests:



src/tests:



The tests themselves are outlined to follow the same structure as their corresponding .py files, whereas integration tests follow the interacting files that are used for the main pipelines for the ML (prepare data set, extract features, generate dataset, train model, test model, predict) and the dashboard (extract features, build dataset, process dataset, main, dashboard).

The CI pipeline that runs automatically upon GitHub commits is located under .github/workflows/ci.yml

3. Test Cases (Everyone)

Integration Tests

Run from src/ML/: python -m pytest tests/integration_tests/ -v

Field	Description
Test Name	

Description	
Preconditions	
Inputs	
Expected Output	
Notes	

Test File: test_extract_to_process.py
Test Case File: extract_features.py and process_dataset.py

Field	**Description**
Test Name	test_all_rows_valid
Description	Verifies that clean data produced by extract_flows() passes through validate_dataset() with no false positives — ensuring the two stages are compatible under normal operating conditions.
Preconditions	make_valid_flow_df() fixture from test_helpers.py file
Inputs	simulate_extract_flows(60) — 60 rows of synthetic but schema-compliant flow data.
Expected Output	0 rows flagged is_valid == False. All rows pass validation without error.
Notes	

Field	**Description**
Test Name	test_invalid_ip_flagged
Description	Confirms that a malformed IP address injected into the extract output is caught by validate_dataset(), preventing bad data from silently reaching downstream stages
Preconditions	make_valid_flow_df() fixture from test_helpers.py file. Row 0 src_ip mutated after fixture creation.

Inputs	simulate_extract_flows(5) with row 0 src_ip set to '999.999.999.999'
Expected Output	result.loc[0, 'is_valid'] == False. error_reason contains 'src_ip' or 'invalid'
Notes	

Field	**Description**
Test Name	test_mismatched_protocol_name_flagged
Description	Checks that the cross-validation logic in validate_row() catches a protocol number/name mismatch (protocol=6 labelled as 'UDP'), which could arise from a mapping bug in extract_features.
Preconditions	Base fixture with protocol=6 (TCP) on all rows. Row 1 protocol_name overwritten.
Inputs	simulate_extract_flows(5) with row 1 protocol_name set to 'UDP'.
Expected Output	result.loc[1, 'is_valid'] == False. error_reason contains 'mismatch'.
Notes	

Field	**Description**
Test Name	test_duplicate_flows_flagged
Description	Validates that exact duplicate rows — same 5-tuple and first_packet_index — are detected and flagged by validate_dataset()
Preconditions	Base 5-row fixture. An exact copy of row 0 is appended, producing 6 rows.
Inputs	pd.concat of simulate_extract_flows(5) with a duplicate of row 0 appended.
Expected Output	At least 2 rows have error_reason containing 'Duplicate'.
Notes	

Test File: test_process_to_build.py
Test Case File: process_dataset.py and build_dataset.py

Field	Description
Test Name	test_invalid_rows_excluded_from_ml
Description	Verifies that build_dataset() only processes rows validated as valid by process_dataset()
Preconditions	make_valid_flow_df(60) with first 10 rows corrupted; validate_dataset() run before build_dataset().
Inputs	60-row DataFrame with rows 0–9 having src_ip='999.0.0.1'. Passed through validate_dataset() then build_dataset().
Expected Output	anomaly_info['total_flows'] equals the number of rows where is_valid == True (50).
Notes	

Field	Description
Test Name	test_anomaly_info_keys_present
Description	Confirms that build_dataset() returns an anomaly_info dict with anomaly_count, total_flows, and anomaly_percentage.
Preconditions	Fully validated 60-row DataFrame from the validated() helper.
Inputs	validated(60) passed directly to build_dataset().
Expected Output	anomaly_count, total_flows, anomaly_percentage in the returned dict
Notes	

Field	Description
Test Name	test_numeric_df_has_no_string_columns
Description	Ensures that all non-numeric columns (IPs, protocol names, validation metadata) are dropped before the DataFrame is passed to IsolationForest

Preconditions	Fully validated 60-row DataFrame, no manual mutations.
Inputs	validated(60) passed to build_dataset(); all columns of numeric_df inspected.
Expected Output	No columns with non-numeric dtype remain, except the boolean 'anomaly' column added by build_dataset().
Notes	

Field	**Description**
Test Name	test_nan_values_filled_before_scaling
Description	Checks that NaN values in numeric columns are filled (e.g. with the column median) before IsolationForest is fitted
Preconditions	Fully validated 60-row DataFrame. One cell set to NaN after validation.
Inputs	validated(60) with row 5 packet_count manually set to np.nan, then passed to build_dataset().
Expected Output	No exception raised. anomaly_info['total_flows'] > 0.
Notes	

Test File: test_full_pipeline.py
Test Case File: extract_features.py, process_dataset.py and build_dataset.py

Field	**Description**
Test Name	test_pipeline_returns_three_values
Description	Smoke test confirming the full pipeline runs end-to-end and returns the three-tuple (flows_df, numeric_df, anomaly_info) that dashboard.py destructures.
Preconditions	make_valid_flow_df(60) available. All three pipeline modules importable.
Inputs	run_core_pipeline(make_valid_flow_df(60)) — clean 60-row input through the full chain.

Expected Output	Result is a 3-tuple. No exception raised.
Notes	

Field	**Description**
Test Name	test_invalid_rows_excluded_from_numeric_df
Description	End-to-end check that rows flagged invalid early in the pipeline (by process_dataset) do not appear in the numeric_df that reaches build_dataset, verifying the filter propagates correctly across all stages.
Preconditions	make_valid_flow_df(60) with 5 rows set to loopback IP before the pipeline runs.
Inputs	60-row DataFrame with rows 0–4 src_ip='127.0.0.1' passed to run_core_pipeline().
Expected Output	anomaly_info['total_flows'] equals the count of is_valid == True rows in flows_df.
Notes	

Field	**Description**
Test Name	test_numeric_df_has_anomaly_column
Description	Verifies that the 'anomaly' boolean column added by build_dataset() is present in so the dashboard scatter plot can render correctly.
Preconditions	Clean 60-row fixture. No mutations.
Inputs	run_core_pipeline(make_valid_flow_df(60)).
Expected Output	'anomaly' column present in numeric_df.
Notes	

Field	Description
Test Name	test_udp_flows_pass_through_pipeline
Description	Regression test confirming that UDP flows (protocol=17) are not incorrectly invalidated by the protocol cross-check in process_dataset when protocol and protocol_name are consistently set.
Preconditions	All rows in the fixture have protocol=17 and protocol_name='UDP' set before the pipeline runs.
Inputs	make_valid_flow_df(60) with protocol and protocol_name overwritten to 17 / 'UDP' throughout.
Expected Output	At least one row has is_valid == True in flows_df. anomaly_info['total_flows'] > 0.
Notes	

Test File: test_pipeline_ml_integration.py
Test Case File: extract_features_unified.py, predict_action_type.py and main.py

Field	Description
Test Name	test_ml_features_df_has_all_model_feature_columns
Description	Confirms that the DataFrame produced by extract_features_unified contains every feature column the trained model expects, catching any column rename or removal at the extraction boundary.
Preconditions	make_ml_features_df() fixture available. ML model and model_features list importable from predict.py.
Inputs	make_ml_features_df(5) — 5 rows with all 13 model feature columns plus 5-tuple merge columns.
Expected Output	missing == []. No column listed in model_features is absent from ml_features_df.
Notes	

Field	Description

Test Name	test_predict_action_type_labels_are_known
Description	Validates that every label returned by predict_action_type() belongs to the set the model was trained on, detecting a mismatch between the saved model . and the label encoder.
Preconditions	make_ml_features_df(5) fixture. Trained model and label encoder files present at expected paths.
Inputs	make_ml_features_df(5) passed to predict_action_type().
Expected Output	set(result['action_type']) - EXPECTED_LABELS == set(). No unknown labels returned.
Notes	

Field	**Description**
Test Name	test_row_count_unchanged_after_left_merge
Description	Ensures the left merge in main.py that attaches action_type predictions to validated_data does not add or drop any rows, preserving the full flow record for the dashboard.
Preconditions	make_validated_df_with_youtube(5, 10) and make_ml_features_df(5) fixtures. _run_merge() replicates the exact merge logic from main.py.
Inputs	validated_df with 15 rows (5 YouTube + 10 background); ml_df with 5 matching rows. Left-merged on 5-tuple.
Expected Output	len(merged) == 15. Row count unchanged after merge.
Notes	

Unit Tests

ML

Test File: test_predict.py
Test Case File: predict.py

Field	Description
Test Name	test_predict_action_type
Description	Verifies predictions are generated and added as action_type column
Preconditions	Mock model, label encoder, and feature list using monkeypatch
Inputs	Valid feature DataFrame with model feature columns
Expected Output	DataFrame returned with non-null action_type column of same length
Notes	

Test File: test_train_model.py
Test Case File: train_model.py

Field	Description
Test Name	test_train_model
Description	Verify that ML training pipeline can train a RandomForest model on a dataset, producing valid predictions and labels are handled correctly
Preconditions	Mock dataset with required feature columns
Inputs	Features: duration, std_iat, avg_iat, pk_count, avg_inbound_size, avg_outbound_size Labels: action values (Play, Like, Comment)
Expected Output	Trained model, LabelEncoder, Prediction (y_pred) for each row in dataset
Notes	The accuracy of predictions doesn't matter here; mock dataset is smaller and doesn't need to match, Monkeypatch used for test isolation

Test File: test_test_model.py
Test Case File: test_model.py

Field	**Description**
Test Name	test_no_predict_proba
Description	Ensures that run_tests() correctly handles models that do not implememt predict_proba method
Preconditions	Dummy dataset generated using the sample_data. A DummyClassifier model trained on the dataset.
Inputs	X_train, X_test, y_train, y_test, original dataframe df and LabelEncoder
Expected Output	run_tests() executes successfully without crashing even though the model lacks a predict_proba method
Notes	

Field	**Description**
Test Name	test_unknown_label_handling
Description	Verifies that run_tests() correctly detects & reports unseen labels when the label encoder encounters an unknown label
Preconditions	A trained LogisticRegression model and a LabelEncoder fitted on the dataset labels. The test captures console output using the capsys fixture
Inputs	X_train, X_test, y_train, y_test, dataframe df and LabelEncoder
Expected Output	Prints a confirmation message indicating that unknown label handling works correctly.
Notes	

Field	**Description**
Test Name	test_confusion_matrix_shape

Description	Ensure confusion matrix generated from model predictions has the correct dimensions (number of classes in dataset)
Preconditions	Trained Logistic Regression model and dummy dataset
Inputs	X_test, y_test, LabelEncoder
Expected Output	Confusion matrix shape equals/matches (number_of_classes, number_of_classes)
Notes	

Field	**Description**
Test Name	test_permutation_importance_effect
Description	Checks that permutation importance is calculated for all features in the dataset
Preconditions	Dummy dataset and a trained RandomForestClassifier model
Inputs	Test feature set (X_test) and labels (y_test)
Expected Output	The number of importance scores returned must equal the number of features in the dataset
Notes	

Test File: test_generate_dataset.py

Test Case File: generate_dataset.py

Field	**Description**
Test Name	test_highest_bytes_flow_labelled_play
Description	Ensures the flow with highest total_bytes is labeled as "Play".

Preconditions	DataFrame with multiple flows.
Inputs	label_flows(df, "Play")
Expected Output	Row with highest total_bytes has action = "Play"; others = "Background".
Notes	

Field	Description
Test Name	test_highest_outbound_ratio_gets_action_label
Description	Ensures highest outbound ratio flow receives selected non-Play action.
Preconditions	DataFrame with varying outbound_ratio.
Inputs	label_flows(df, "Like")
Expected Output	Highest outbound_ratio row labeled "Like".
Notes	

Field	Description
Test Name	test_csv_written_when_data_exists
Description	Confirms CSV file is generated when data is available.
Preconditions	Temporary path fixture (tmp_path).
Inputs	df.to_csv(output)

Expected Output	CSV file exists in temporary directory.
Notes	

Field	Description
Test Name	test_run_returns_dataframe_on_success
Description	Verifies that run() returns a DataFrame when PCAP files are processed successfully.
Preconditions	Mocked glob and extract_ml_features functions.
Inputs	run(data_dir="datasets/", actions=["Like"])
Expected Output	Function returns non-null Pandas DataFrame.
Notes	

Main Pipeline

Test File: test_dashboard.py

Test Case File: dashboard.py

Field	Description
Test Name	test_known_columns_are_renamed
Description	Verifies that predefined internal column names are correctly renamed to user-friendly labels using rename_cols().

Preconditions	DataFrame created with known column names.
Inputs	Pandas DataFrame with columns: src_ip, dst_ip, byte_count.
Expected Output	Columns renamed to: Source Address, Destination Address, Byte Count.
Notes	

Field	Description
Test Name	test_invalid_rows_are_removed
Description	Confirms that rows where is_valid == False are filtered out.
Preconditions	DataFrame created using make_flow_df() containing both valid and invalid rows.
Inputs	DataFrame filtered using df[df["is_valid"] == True].
Expected Output	Resulting DataFrame contains only rows where is_valid == True.
Notes	

Field	Description
Test Name	test_combined_filters
Description	Ensures multiple filters applied together correctly narrow the dataset.
Preconditions	Valid DataFrame loaded.
Inputs	Combined filter on protocol_name and src_ip.

Expected Output	Only rows matching both conditions are returned.
Notes	

Field	Description
Test Name	test_anomalous_rows_filtered_correctly
Description	Verifies only rows marked as anomaly=True are selected.
Preconditions	DataFrame with one anomalous flow.
Inputs	Filter: df[df["anomaly"] == True].
Expected Output	Exactly one anomalous row returned.
Notes	

Test File: test_extract_features_unified.py
Test Case File: extract_features_unified.py

Field	Description
Test Name	test_is_google_youtube_ip_logic
Description	Verifies the system correctly determines between 'Youtube' related IP addreses and other IP addreses
Preconditions	
Inputs	IP addreses: "172.217.1.1" , "1.1.1.1"
Expected Output	"172.217.1.1" returns true and "1.1.1.1" returns false.
Notes	

Field	Description
Test Name	test_unified_extraction_logic
Description	Tests that one pass over packets correctly populates both the standard dashboard flows and the ML features.
Preconditions	
Inputs	Mock PCAP file containing 3 packets, making up 2 flows.
Expected Output	Number of flows = 2 For flows identified as a Youtube flow: packet count = 2, byte count = 300, protocol_name = TCP. For ML extraction: dataframe length – 1, src_ip = input IP, packet count -= 2, duration = 1, average outbound size = 150
Notes	

Field	Description
Test Name	test_ml_only_flag
Description	Verifies the ml_only parameter skips dashboard flow processing.
Preconditions	
Inputs	"192.168.1.5", "172.217.0.1", 12345, 443, 6, 100, 1000.0, 0 as: src, dst, sport, dport, proto, size, ts, index, layer="TCP"
Expected Output	General extract features dataframe is empty, ML features dataframe contains data
Notes	

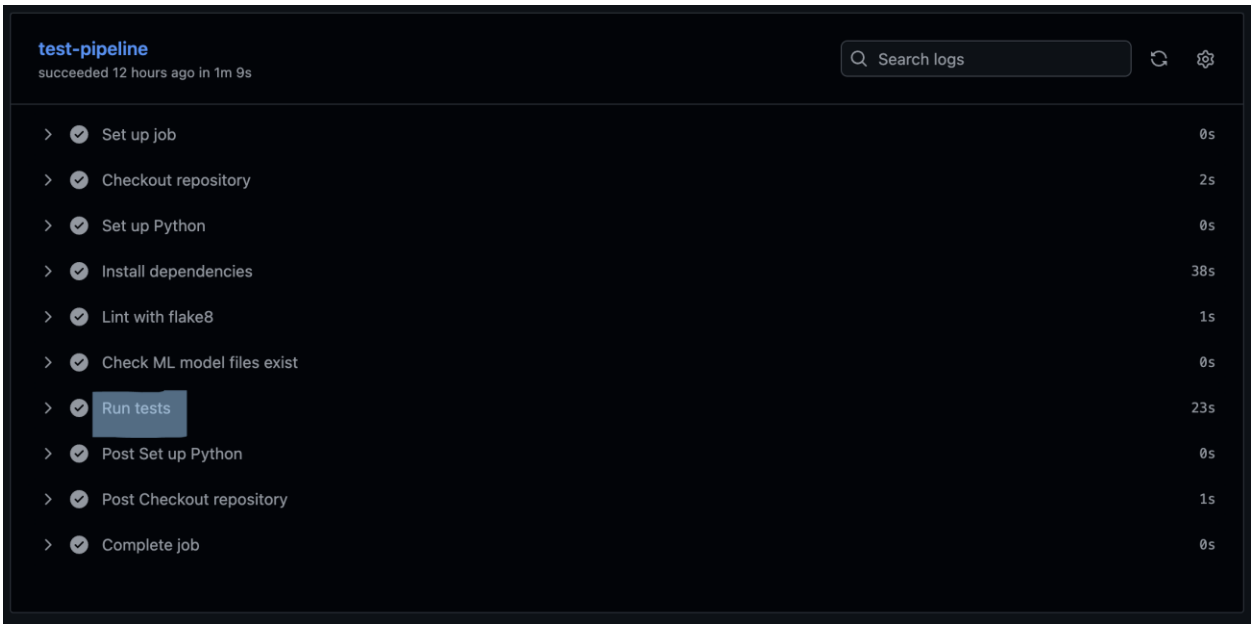
4. Test Execution (Sophia)

Requirements:

- pytest and python (included in requirements.txt)

Instructions:

- Run ”pytest” command from SourceCode/network-traffic-profiler
- Run “pytest test_filename.py” for single tests
- GitHub Actions Workflow -> automated tests upon pushing



5. Test Coverage (Everyone)

- Describe what percentage of code is covered.
- Optional: include a diagram or table showing which modules/functions are tested.

Tomiris
Coverage measured by pytest-cov

Test File	Statements	Miss	Coverage
test_dashboard.py	163	0	100%
test_generate_dataset.py	139	3	98%
test_extract_to_process.py	64	0	100%
test_full_pipeline.py	74	2	97%
test_pipeline_ml_integration.py	91	1	99%
test_process_to_build.py	67	1	99%

Sophia
Coverage measured by pytest-cov

Test File	Statements	Miss	Coverage
test_train_model.py			
test_predict.py			

6. Known Issues / Limitations (Everyone)

Sophia:

- Action classification is being tested in an additional script called accuracy_on_dataset.py, although it is not completely synchronised with the actual dashboard results.
- Anomaly detection accuracy is not being tested, as it is not the focus of this project.

Tomiris:

- Streamlit UI components (st.sidebar, st.plotly_chart, st.dataframe, etc.) cannot be tested with pytest. Tests only cover the pure-Python data logic, not what is visually rendered.
- The test suite does not validate that master_training_data.csv is written to the correct production directory — only that the CSV write mechanism works in isolation using tmp_path.
- The anomaly detection logic in main.run_pipeline() is not unit-tested here. The tests only verify that the dashboard correctly displays whatever anomaly_info is returned.
- ML integration tests depend on the trained model .pkl and label encoder files being present at runtime — if these are missing, the entire TestExtractToPredict and TestPredictToMerge classes will fail
- make_valid_flow_df() generates synthetic data only — edge cases present in real PCAP captures (e.g. fragmented packets, malformed headers) are not represented in the test fixtures

7. Maintenance / Adding New Tests (Sophia)

Guidelines for contributing to new test cases

1. Use BDD (behaviour driven testing) approach, defining user stories and requirements first and then writing corresponding tests
2. Write tests for functions following the original script order
3. Continuously check functionality with pytest
4. Ensure all imports are reached from src or src/ML
5. Do not commit broken tests to main branch
6. Check CI pipeline after committing and fix broken tests if needed

Recommended naming conventions and folder structure

- Filename: test_file.py, e.g. test_main.py
- Anything regarding ML goes into the ML/tests folder
- Anything regarding the main pipeline goes into the /src folder

Test Documentation Template

Use the following template to document new tests:

Test File: test_file.py
Python File: file.py

Field	Description
Test Name	Unique and descriptive name
Description	What the test checks
Preconditions	Any setup required (mock data, test database state, configs)
Inputs	Data or parameters used in the test
Expected Output	What result or behavior is expected
Notes	Additional notes

- Aim for a maximum of 100 words per table when selecting everything